

PUDUCHERRY RTO MODERNIZATION PORTAL

Comprehensive System Audit & New AI Proctoring Architecture Specification

Full Portfolio Project Project Documentation

1. Executive Summary & Architecture Context

The current digital infrastructure for the Puducherry Regional Transport Office (RTO) functions via a hybrid model. The local state portal operates primarily as a static informational index, while all core, transactional user workflows are executed via redirections to the central government's National **Parivahan Sewa** portal (Sarathi for licensing, Vahan for vehicle logistics). This report documents current feature efficacy and details the structural plans for a completely modernized portal.

The Redirection UX Trap: The primary systemic friction point stems from design and session discontinuity. Users initiating local requests are transferred unexpectedly to the massive national infrastructure, disrupting user flows and reducing application trust markers.

2. Current Feature Performance & Usability Matrix

Core System Module	Current Performance	Usability Score	Observed Technical/UX Friction
Driving Licence Services (Sarathi Integration)	MODERATE Stable infrastructure but prone to peak hour load-shedding.	POOR Dense, text-heavy legalities and complex form validation.	Booking a physical test slot relies on restrictive multi-layered calendar popups and redundant OTP verifications.
Vehicle Registration & RC Logistics (Vahan Integration)	SLOW TO MODERATE Heavy data pulling leads to frequent gateway timeouts.	POOR Cluttered interfaces demanding high inputs.	Requires manual entries of long-character chassis/engine strings for simple status tracking; lacks a step-by-step physical delivery tracker.
Fee Payments & Fancy Number Bidding	UNRELIABLE Payment gateway handshakes intermittently fail.	FAIR Unresponsive multi-row tables.	Financial metrics, taxes, and premium bids are formatted as static HTML grids or legacy PDFs, creating severe barriers on mobile devices.
Commercial Permits & Fitness Clearance	MODERATE Handles plain textual inputs reliably.	POOR Rigid constraints and zero proactive alerts.	Extremely strict document upload payloads cause silent file drops. Zero automation to notify operators regarding upcoming lifecycle updates.
Local Directory & Regional Fare Index	GOOD Static files load efficiently due to low weight.	OUTDATED Circa 2010 web design aesthetic.	Regional RTO codes (PY-01 through PY-05) and current auto/bus fares are locked inside deeply buried, un-stylized text frames.

3. Future General Feature Overhaul

- **Unified Dashboard via Single Sign-On (SSO):** Authentication through DigiLocker or Aadhaar to automatically ingest and render active licenses, current registration certificates, and traffic challans immediately on a sleek dashboard.
- **Visual Status Pipeline:** Replacement of cryptic state labels with an intuitive horizontal pipeline tracking (e.g., Document Submission → Verification Queue → Inspector Check → Printing → Dispatch).

- **Interactive Fee Engine:** Reactive UI widgets matching parameters like Vehicle Class, Fuel Type, and Ex-Showroom Valuation to accurately render road taxes, eliminating static PDFs.
-

4. Module Addendum: AI-Powered Learner's Licence (LL) Exam Specification

To create a competitive edge for this portfolio build, the redesigned application incorporates a real-time smart proctoring system for the automated Learner's Licence testing portal. The design architecture below treats anti-cheating protocols as **deterrence and anomaly tracking state machines** rather than unrealistic browser hardware blocks.

4.1 Relational Data Model (Prisma Schema syntax)

```
model UserBiometricProfile {
  id          String    @id @default(uuid())
  userId      String    @unique
  user        User      @relation(fields: [userId], references: [id])
  faceEmbedding Json     // Float32Array(128) stringified into a JSON array
  capturedAt  DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}

enum ViolationType {
  FACE_MISMATCH
  NO_FACE_DETECTED
  MULTIPLE_FACES
  TAB_SWITCH
  FULLSCREEN_EXIT
}

model ExamSession {
  id          String    @id @default(uuid())
  userId      String
  examType    String    // e.g., "Puducherry_LL_Mock_01"
  status      String    // "ONGOING", "COMPLETED",
  "TERMINATED_AUTOMATICALLY"
  score       Int        @default(0)
  startedAt   DateTime   @default(now())
  endedAt     DateTime?
  violations   ExamViolationLog[]
}

model ExamViolationLog {
  id          String    @id @default(uuid())
  examSessionId String
  examSession ExamSession @relation(fields: [examSessionId], references: [id])
  type        ViolationType
  timestamp   DateTime   @default(now())
  evidenceUrl String?     // Cloud bucket pointer to captured frame snapshot
  severityWeight Int      @default(1)
}
```

4.2 The Violation Scoring State Machine

The frontend tracker handles browser events and facial embeddings dynamically using a continuous grace period algorithm to prevent false flags from bad environmental illumination:

- **Immediate Violations:** Evading mandatory Fullscreen or switching windows logs an immediate penalty value score of **3**.
- **Biometric Validation:** Webcam snapshots are processed every 3 seconds. Continuous matching exceptions (No Face / Face Mismatch) require 3 sequential failures before pushing an active violation penalty of **2**.
- **Termination Condition:** If the total session severity score calculations match or exceed **5**, the engine calls a background mutation API to immediately terminate input capability and mark the test session status as corrupted.

4.3 Front-end React Execution Script Blueprint

```

import React, { useEffect, useRef, useState } from 'react';
import * as faceapi from 'face-api.js';

interface LLProctorPanelProps {
  referenceEmbedding: number[];
  onViolationTriggered: (type: string, details: string) => void;
  onExamTerminated: () => void;
}

export const LLProctorPanel: React.FC<LLProctorPanelProps> = ({
  referenceEmbedding,
  onViolationTriggered,
  onExamTerminated,
}) => {
  const videoRef = useRef<HTMLVideoElement | null>(null);
  const [modelsLoaded, setModelsLoaded] = useState(false);
  const consecutiveMismatches = useRef(0);
  const totalViolationScore = useRef(0);

  useEffect(() => {
    const loadModels = async () => {
      const MODEL_URL = '/models';
      await Promise.all([
        faceapi.nets.ssdMobilenetv1.loadFromUri(MODEL_URL),
        faceapi.nets.faceLandmark68Net.loadFromUri(MODEL_URL),
        faceapi.nets.faceRecognitionNet.loadFromUri(MODEL_URL),
      ]);
      setModelsLoaded(true);
    };
    loadModels();
  }, []);

  useEffect(() => {
    if (!modelsLoaded) return;
    navigator.mediaDevices.getUserMedia({ video: {} })
      .then((stream) => { if (videoRef.current) videoRef.current.srcObject =
stream; })
      .catch((err) => console.error("Webcam stream access denied:", err));
  }, [modelsLoaded]);

  useEffect(() => {
    if (!modelsLoaded) return;
    const proctorInterval = setInterval(async () => {
      if (!videoRef.current) return;
      const detection = await faceapi
        .detectSingleFace(videoRef.current, new faceapi.SsdMobilenetv1Options())
        .withFaceLandmarks()

```

```

        .withFaceDescriptor());

    if (!detection) {
        consecutiveMismatches.current += 1;
        if (consecutiveMismatches.current >= 3) handleViolation('NO_FACE_DETECTED',
1);
        return;
    }

    const referenceDescriptor = new Float32Array(referenceEmbedding);
    const currentDescriptor = detection.descriptor;
    const distance = faceapi.euclideanDistance(referenceDescriptor,
currentDescriptor);

    if (distance > 0.60) {
        consecutiveMismatches.current += 1;
        if (consecutiveMismatches.current >= 3) handleViolation('FACE_MISMATCH', 2);
    } else {
        consecutiveMismatches.current = 0;
    }
}, 3000);
return () => clearInterval(proctorInterval);
}, [modelsLoaded, referenceEmbedding]);

const handleViolation = (type: string, scoreWeight: number) => {
    totalViolationScore.current += scoreWeight;
    onViolationTriggered(type, `Cumulative threshold penalty score adjustment: $
{scoreWeight}`);
    if (totalViolationScore.current >= 5) onExamTerminated();
};

return (
    <div style={{ position: 'fixed', bottom: 20, right: 20, zIndex: 999 }}>
        {!modelsLoaded ? (
            <p style={{ color: 'orange' }}>Initializing Biometric Shield...</p>
        ) : (
            <video ref={videoRef} autoPlay muted style={{ width: '200px', height: '150px',
borderRadius: '8px' }} />
        )}
    </div>
);
};

```